

Fiche TP — Semaine 4

La mise en forme du MiniForum

Développement web, classe passerelle

Durée 4h

Rendu avant S5

Objectif du TP

Écrire la **première feuille de style** du MiniForum : typographie, palette de couleurs et espacements pour l'ensemble du site, puis construire l'en-tête et la liste des sujets avec **Flexbox**.

Avant de commencer

Ce TP suppose que les notions du polycopié *S04_polycop* (sélecteurs, cascade et spécificité, héritage, modèle de boîte, box-sizing, Flexbox, pseudo-classes) ont été vues en cours. Garde-le à portée de main.

Tu travailles dans le dépôt miniforum, avec les quatre pages produites en semaines 2 et 3 : `index.html`, `sujet.html`, `inscription.html`, `connexion.html`.

Comment lire cette fiche

Comme les semaines précédentes, cette fiche ne donne pas le code à recopier. Elle décrit le **rendu attendu** sous forme de maquette et précise ce qu'il faut produire. Les quelques extraits de CSS présents sont des **points de départ** à compléter, pas des solutions.

Les couleurs et les tailles indiquées sont des propositions : tu peux les changer, à condition de respecter les contraintes de contraste de l'étape 7.

Étape	Ce que tu produis	Temps
1–2	Brancher et poser les fondations	un site déjà lisible
3	Poser les classes dans le HTML	un HTML prêt à styler
4	L'en-tête en Flexbox	une vraie barre de navigation
5	Les cartes de sujet en Flexbox	la page d'accueil du forum
6	Les formulaires	quatre pages cohérentes
7	États, focus et contrastes	un site utilisable par tous
8	Relecture, validation et envoi	un dépôt propre

1 Brancher la feuille de style

1. Ouvrir le dossier miniforum dans VS Code et récupérer les éventuelles mises à jour :

```
git pull
```

2. Vérifier que le fichier `style.css` créé en semaine 1 est bien là, à la racine du projet. S'il a disparu, le recréer, vide.

3. Dans le `<head>` des **quatre** pages, ajouter la liaison :

```
<link rel="stylesheet" href="style.css">
```

4. Écrire une règle de test dans `style.css`, par exemple `body { background-color: pink; }`, ouvrir les quatre pages avec *Live Server* et vérifier qu'elles rosissent toutes.
5. Supprimer la règle de test.

Si rien ne change

Trois causes, dans l'ordre de fréquence :

- le `href` ne pointe pas au bon endroit — `style.css` est-il bien à côté du fichier HTML ?
- le `<link>` est en dehors du `<head>` ;
- le navigateur sert une ancienne version du fichier : recharge avec `Ctrl + Maj + R`.

L'onglet *Réseau* des outils de développement tranche en deux secondes : si `style.css` apparaît en 404, c'est le chemin.

2 Poser les fondations

Cette étape ne touche à aucun élément en particulier : elle définit le socle dont tout le reste héritera.

Le squelette du fichier

Avant d'écrire la moindre règle, structure `style.css` en six sections commentées. Tu écriras chaque règle **dans la bonne section**, du début à la fin du TP.

```

1  /* === 1. Reglages generaux ===== */
2
3  /* === 2. En-tete et navigation ===== */
4
5  /* === 3. Contenu principal ===== */
6
7  /* === 4. Cartes de sujet ===== */
8
9  /* === 5. Formulaires ===== */
10
11 /* === 6. Pied de page ===== */

```

Extrait 1 – À placer tel quel en tête de `style.css`

Les réglages généraux

À produire dans la section 1

- la règle universelle `*{ box-sizing: border-box; }` ;
- sur `body` : une pile de polices sans empattement, une taille de 16px, un `line-height` de 1.6, une couleur de texte foncée et une couleur de fond claire ;
- une remise à zéro de la marge par défaut du `body` ;
- sur `main` : une largeur maximale d'environ 900px, centrée par `margin: 0 auto`, et un `padding` intérieur.

Choisis ta palette maintenant, note-la en commentaire

Quatre à cinq couleurs suffisent, et il vaut mieux les fixer une fois que les improviser règle après règle. Une proposition, à reprendre ou à remplacer :

Rôle	Valeur	Employée pour
Texte	#1A1A1A	le texte courant
Accent	#1D4E89	liens, boutons, titres
Fond de page	#F2F2F2	le body
Fond de carte	#FFFFFF	cartes et formulaires
Bordure	#DDDDDD	traits et séparateurs

Note-les en commentaire en haut du fichier. En **semaine 5**, ces cinq lignes deviendront des variables CSS et ne seront plus jamais recopiées.

Regarde la page maintenant

Avec une dizaine de déclarations, le site est déjà plus lisible qu'il ne l'a jamais été : texte aéré, colonne de largeur raisonnable, contraste correct.

C'est le meilleur rapport effort/résultat de tout le cours, et c'est la raison pour laquelle on part du général avant de traiter le moindre composant.

Point d'étape

```
git add .
git commit -m "Feuille de style branchée, réglages généraux"
```

3 Poser les classes dans le HTML

C'est la **seule** retouche du HTML de la semaine, et elle n'ajoute aucune balise : uniquement des attributs class sur des éléments qui existent déjà.

Classe	Sur quel élément	Dans quelle page
entete	le <header>	les quatre
liste-sujets	la liste des sujets	index.html
carte-sujet	chaque sujet de la liste	index.html
carte-message	chaque message affiché	sujet.html
bouton	chaque <button type="submit">	les quatre
pied	le <footer>	les quatre

À produire

Les six classes ci-dessus placées dans les quatre fichiers HTML. Aucune balise ajoutée, aucune balise supprimée, aucun attribut style.

Nommer par le rôle, pas par l'apparence

carte-sujet dit ce qu'est l'élément. boîte-blanche dirait à quoi il ressemble aujourd'hui — et deviendrait faux à la première modification.

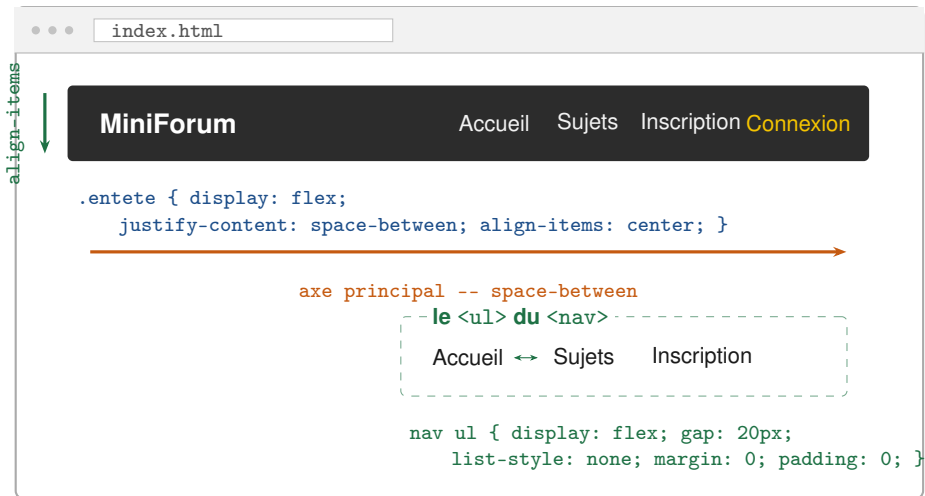
Minuscules, tiret entre les mots, pas d'accent. Ces noms te suivront jusqu'à la soutenance : JavaScript s'en servira dès la semaine 7 pour retrouver les éléments.

Ne remplace pas tes balises sémantiques par des `<div>`

Il peut être tentant de « simplifier » le HTML pour styler plus facilement. Le `<header>`, le `<nav>`, le `<main>`, les `<article>` et le `<footer>` de la semaine 2 restent en place : une classe s'ajoute **sur** la balise sémantique, elle ne la remplace pas.

4 L'en-tête et la navigation en Flexbox

Le titre du site à gauche, le menu à droite, le tout centré verticalement et sur une seule ligne : c'est le cas d'école de Flexbox.



À produire dans la section 2 de style.css

- sur `.entete` : `display: flex`, le titre et le menu écartés aux deux extrémités, un alignement vertical centré, un fond sombre, un texte clair et un padding confortable ;
- sur le `<ul>` du `<nav>` : `display: flex`, un gap d'environ 20px, les puces retirées, et les `margin` / `padding` par défaut du navigateur remis à zéro ;
- sur les liens de la navigation : couleur claire, soulignement retiré ;
- sur `.pied` : un fond discret, un texte centré et un padding vertical.

Le `<ul>` a des marges que tu n'as pas écrites

Tout navigateur applique au `<ul>` une marge verticale et un padding gauche par défaut, qui décalent le menu et cassent l'alignement.

C'est le cas typique où `margin: 0; padding: 0;` est nécessaire — non pas pour « faire joli », mais pour reprendre la main sur un style que tu n'as pas choisi. Les outils de développement le montrent : la règle apparaît comme provenant de la feuille de style de l'agent utilisateur.

Vérifie que tu utilises bien Flexbox

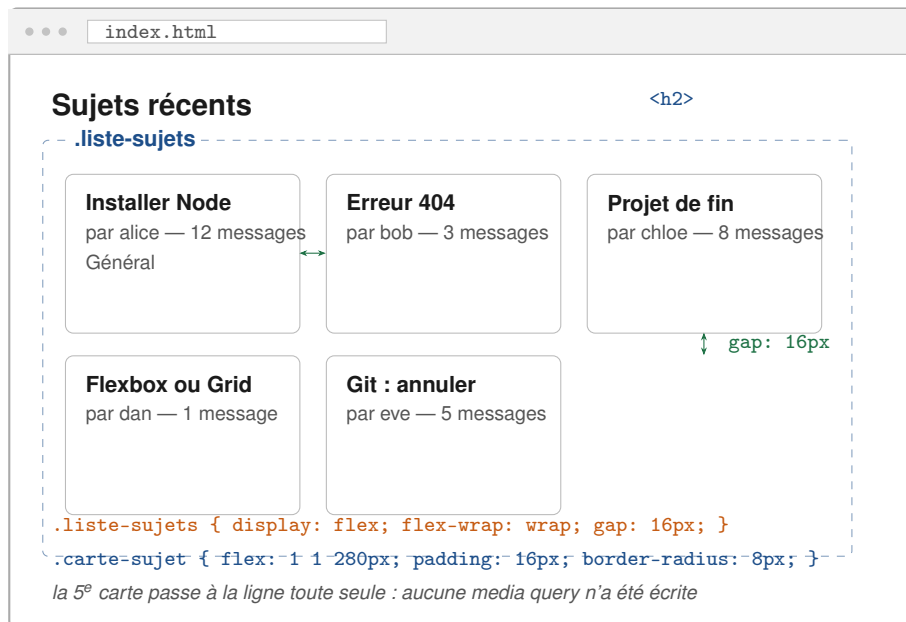
Un menu peut sembler correct pour de mauvaises raisons — des `<li>` passés en `inline-block`, par exemple. Dans les outils de développement, un badge `flex` apparaît à côté de tout élément dont le `display` vaut `flex`. Clique dessus : le navigateur dessine les axes et les espacements sur la page.

Point d'étape

```
git add .
git commit -m "En-tete et navigation en Flexbox"
```

5 Les cartes de sujet en Flexbox

La liste des sujets de `index.html` devient une série de cartes de largeur souple, qui passent à la ligne quand la place manque.



À produire dans la section 4 de `style.css`

- sur `.liste-sujets` : `display: flex`, le passage à la ligne autorisé, un gap d'environ 16px, et — si la liste est un `<ul>` — les puces et les marges par défaut retirées ;
- sur `.carte-sujet` : `flex: 1 1 280px`, un fond blanc, une bordure fine, des coins arrondis et un padding intérieur ;
- à l'intérieur d'une carte : un titre plus grand et coloré, des métadonnées (auteur, nombre de messages) plus petites et grises ;
- sur `.carte-message` dans `sujet.html` : la même présentation de carte, empilée verticalement.

Le test à faire immédiatement

Réduis la largeur de la fenêtre progressivement. Les cartes doivent **rétrécir**, puis **passer à la ligne** vers 280 pixels de large, sans jamais déborder ni provoquer de barre de défilement horizontale.

Si elles débordent : il manque `flex-wrap: wrap`, ou `box-sizing: border-box` n'a pas été posé au début du fichier.

Comprendre `flex: 1 1 280px`

Les trois valeurs sont `flex-grow`, `flex-shrink` et `flex-basis` : la carte **part** de 280 pixels, **peut grandir** pour remplir la ligne, et **peut rétrécir** si nécessaire.

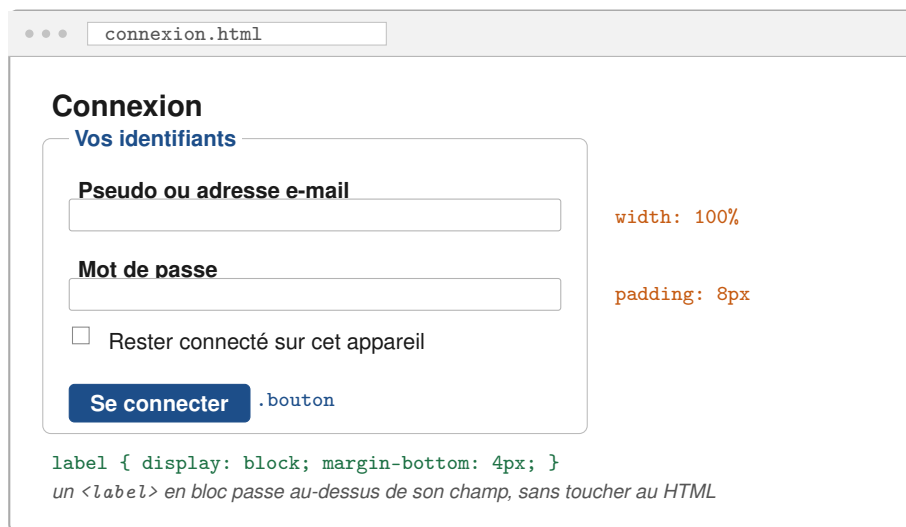
Essaie `flex: 0 0 280px` pour comparer : les cartes gardent alors une largeur fixe et laissent un vide à droite. Regarde les deux, puis choisis en connaissance de cause.

Point d'étape

```
git add .
git commit -m "Cartes de sujet en Flexbox"
```

6 Les formulaires

Les quatre formulaires de la semaine 3 sont fonctionnels et illisibles : champs collés, largeurs inégales, police différente du reste du site.

**À produire dans la section 5 de style.css**

- rendre la police héritée aux champs : `input, textarea, select, button { font-family: inherit; }`
- passer les `<label>` en `display: block`, avec une petite marge sous chacun ;
- donner aux champs une largeur de 100%, un padding d'environ 8px, une bordure fine et des coins légèrement arrondis ;
- espacer les `<fieldset>` entre eux et leur donner un fond de carte ;
- styler `.bouton` : fond de couleur d'accent, texte blanc, pas de bordure, padding généreux, coins arrondis, `cursor: pointer`.

width: 100% sur un champ, sans border-box

Un `<input>` à `width: 100%` avec un padding et une bordure déborde de son `<fieldset>` — sauf si `box-sizing: border-box` a été posé en première ligne du fichier.

Si un champ dépasse, ne cherche pas une largeur en 98% : remonte vérifier la section 1.

Le <textarea> se redimensionne

Ajoute `resize: vertical` au `<textarea>` du formulaire de réponse : l'utilisateur peut l'agrandir en hauteur, mais plus en largeur — ce qui évite qu'il ne défonce la mise en page.

7 États, focus et contrastes

Le site est habillé. Reste à vérifier qu'il reste utilisable par tout le monde — cette étape compte autant que les précédentes.

Les états interactifs

À produire

Pour **chaque** élément interactif du site — liens de navigation, liens dans le texte, boutons, champs de formulaire :

- un état `:hover` visible (changement de couleur, de fond ou de soulignement) ;
- un état `:focus-visible` au moins aussi visible, réalisé avec `outline` et `outline-offset` ;
- pour les champs : un `:focus` qui change la couleur de la bordure.

Aucun `outline: none` dans le fichier

Cherche cette chaîne dans `style.css` (Ctrl + F) : elle ne doit apparaître nulle part. Si un contour par défaut ne te plaît pas, **remplace-le** par le tien, plus visible — ne le supprime pas.

Un site sans contour de focus est inutilisable au clavier, et le point est vérifié au jalon 1.

Le test des trente secondes, version CSS

1. Cliquer dans la barre d'adresse, puis **lâcher la souris**.
2. Parcourir chaque page avec Tab, du début à la fin.
3. À chaque arrêt, vérifier que l'élément actif est **clairement identifiable** — sans deviner.
4. Vérifier que l'ordre suit toujours la lecture naturelle : le CSS de cette semaine ne doit pas l'avoir modifié.

Ce que le CSS peut casser

Trois pratiques déplacent l'ordre visuel sans déplacer l'ordre du clavier, et rendent la navigation incohérente : `order` dans un conteneur flex, `flex-direction: row-reverse`, et le positionnement absolu (vu plus tard).

Aucune n'est nécessaire cette semaine. Si l'ordre doit changer, change le HTML.

Les contrastes

1. Ouvrir les outils de développement, onglet *Accessibilité*, et lire le rapport de contraste de chaque couple texte / fond.
2. Vérifier le seuil **AA** : 4.5 : 1 pour le texte courant, 3 : 1 pour les grands titres.
3. Contrôler en particulier : le texte gris des métadonnées sur fond blanc, les liens de la navigation sur fond sombre, et le texte du bouton sur sa couleur d'accent.

Le sélecteur de couleur des DevTools calcule le contraste

Clique sur le carré de couleur à côté d'une déclaration `color` : le sélecteur affiche le rapport de contraste et trace une ligne au-delà de laquelle le seuil AA n'est plus respecté. Il suffit de remonter au-dessus de cette ligne.

8 Relire, valider et sauvegarder

Relire sa feuille de style

À chercher dans <code>style.css</code>	Pourquoi
<code>!important</code>	Interdit dans le projet : trouve la vraie cause du conflit
<code>outline: none</code>	Rend le site inutilisable au clavier
<code>#</code> en début de sélecteur	Le style passe par les classes, pas par les <code>id</code>
Règles en double	Deux règles pour le même sélecteur : fusionne-les
Règles hors section	Chaque règle appartient à l'une des six sections

Et dans les fichiers HTML

Cherche `style=` dans les quatre pages : il ne doit y avoir **aucun** attribut `style`. Toute mise en forme vit dans `style.css`.

Valider

1. Passer `style.css` au validateur CSS du W3C : jigsaw.w3.org/css-validator, onglet *Par upload de fichier*.
2. Repasser les quatre pages à validator.w3.org — l'ajout des classes ne doit avoir introduit aucune erreur.
3. Ouvrir la console des outils de développement : aucun message d'erreur, en particulier aucun 404 sur `style.css`.

Symptôme fréquent cette semaine	Cause la plus probable
Une règle ne s'applique pas	Une règle plus spécifique gagne : regarde la déclaration barrée dans <i>Styles</i>
Le <code><link></code> ne charge rien	Chemin incorrect : l'onglet <i>Réseau</i> affiche un 404
La déclaration apparaît en grisé	Nom de propriété ou valeur mal orthographié
Deux blocs à 50 % ne tiennent pas côte à côte	<code>box-sizing: border-box</code> absent
La hauteur d'un lien ne change pas	Un <code><a></code> est <code>inline</code> : passer en <code>inline-block</code>
L'espacement vertical est plus petit que prévu	Fusion des marges verticales : c'est normal
Le menu est décalé vers la droite	Le padding par défaut du <code></code>
Rien ne change après modification	Cache du navigateur : Ctrl + Maj + R

Envoyer sur GitHub

```
git add .
git commit -m "Formulaires, etats interactifs et contrastes"
git push
```

Vérifier que c'est bien parti

Sur github.com, ton dépôt doit contenir `style.css` en plus des quatre fichiers HTML, et l'onglet *Commits* doit montrer **plusieurs** commits pour cette séance — un par étape, pas un seul en fin de TP.

9 Pour aller plus loin (optionnel)

Si tu as terminé en avance :

- Ajouter une ombre légère aux cartes (`box-shadow`) et une transition douce au survol (`transition: box-shadow 0.2s`). Compare avec et sans : l'effet est-il utile ou seulement joli ?
- Colorer une ligne de message sur deux avec `:nth-child(even)` dans `sujet.html`.
- Styler les états de validation d'un champ avec `:invalid` — et vérifier que la couleur n'est **jamais** le seul signal d'erreur (semaine 3).
- Mettre en évidence la page courante dans le menu avec une classe `.lien-actif`, et calculer la spécificité de ta règle pour comprendre pourquoi elle gagne (ou pas).
- Passer le `<footer>` en Flexbox avec trois zones, et essayer successivement `space-between`, `space-around` et `space-evenly` pour voir la différence.
- Reconstruire la même page d'accueil avec `flex-direction: column` : quelles propriétés changent-on, et laquelle agit désormais verticalement ?

10 Points de contrôle

Auto-évaluation

- ☐ `style.css` existe et est relié aux **quatre** pages par un `<link>` dans le `<head>`
- ☐ Le fichier est organisé en six sections commentées, et chaque règle est dans la bonne
- ☐ `*{ box-sizing: border-box; }` figure en tête du fichier
- ☐ La typographie et les couleurs sont posées sur `body` et héritées partout
- ☐ Le `<main>` a une largeur maximale et est centré par `margin: 0 auto`
- ☐ Les six classes prévues sont posées dans le HTML, nommées par leur rôle
- ☐ Aucune balise sémantique n'a été remplacée par un `<div>`
- ☐ L'en-tête est en Flexbox : titre à gauche, menu à droite, alignés verticalement
- ☐ Le menu est un `<ul>` en Flexbox, espacé par `gap`, sans puces
- ☐ La liste des sujets est en Flexbox avec `flex-wrap: wrap` et un `gap`
- ☐ Les cartes rétrécissent puis passent à la ligne, sans jamais déborder
- ☐ Les champs de formulaire héritent de la police du site
- ☐ Chaque `<label>` est en `display: block` au-dessus de son champ
- ☐ Chaque élément interactif a un `:hover` **et** un `:focus-visible`
- ☐ Le fichier ne contient ni `!important`, ni `outline: none`
- ☐ Aucun attribut `style` dans les quatre pages HTML
- ☐ Tous les contrastes atteignent le seuil AA (4.5 :1, ou 3 :1 pour les grands titres)
- ☐ Chaque page se parcourt entièrement au clavier, avec un focus visible
- ☐ `style.css` passe le validateur CSS du W3C, et les quatre pages passent `validator.w3.org`
- ☐ Le travail est poussé sur GitHub, en **plusieurs** commits au message clair

À rendre avant la semaine 5

Le dépôt GitHub miniforum à jour, avec `style.css` et les quatre pages validées, contrastées et navigables au clavier.

La semaine 5 ouvre le **responsive** : media queries, CSS Grid, unités relatives et variables CSS. La palette que tu viens de fixer deviendra un jeu de variables, et la mise en page s'adaptera du téléphone à l'écran large — toujours sans toucher au HTML.

Bloqué ?

Si une étape résiste plus de dix minutes, ouvre les outils de développement **avant** de demander : la déclaration barrée du panneau *Styles* répond à la majorité des questions de ce TP. Le polycopié *S04_polycop* contient le reste.